

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ

УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

(МОСКОВСКИЙ ПОЛИТЕХ)

КУРСОВОЙ ПРОЕКТ

по курсу

«Разработка веб-приложений»

ТЕМА

**«Разработка веб-приложения для автоматизации процесса
бронирования столиков в ресторане»**

Выполнила: Антипова Анастасия Максимовна

Группа: 241-3211

Проверил: А. С. Кружалов

Москва, 2026

**«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)**

УТВЕРЖДАЮ
заведующая кафедрой
«Инфокогнитивные технологии»

_____ / Е. А. Пухова /

«____» _____ 2026 г.

ЗАДАНИЕ

на выполнение курсовой работы (проекта)

Антиповой Анастасии Максимовне,

обучающейся группы 241-3211,

направления подготовки 09.03.01 «Информатика и вычислительная техника»

по дисциплине «Разработка веб-приложений»

на тему «Разработка веб-приложения для автоматизации процесса бронирования столиков в ресторане»

1. Исходные данные к работе (проекту): информационные ресурсы в сети интернет, научные публикации в открытой печати.

2. Содержание задания по курсовой работе (проекту) – перечень вопросов, подлежащих разработке:

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Раздел 1. Анализ предметной области			
Задача 1.1. Обзор существующих программных продуктов по теме работы	10	10.04.2026	
Задача 1.2. Анализ программных инструментов разработки веб-приложений	7	13.04.2026	
Задача 1.3. Формулировка цели и задач работы	8	18.04.2026	
Раздел 2. Проектирование веб-приложения			
Задача 2.1. Анализ целевой аудитории	5	25.04.2026	
Задача 2.2. Описание функциональности приложения (диаграмма вариантов использования, user story)	7	30.04.2026	

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Задача 2.3. Проектирование модели данных (ER-диаграмма, логическая и физическая схемы БД)	8	05.05.2026	
Задача 2.4. Разработка макетов страниц (Wireframe)	5	16.05.2026	
Раздел 3. Разработка веб-приложения			
Задача 3.1. Разработка базовой структуры приложения и вёрстка шаблонов страниц	8	18.05.2026	
Задача 3.2. Реализация аутентификации пользователей	7	25.05.2026	
Задача 3.3. Реализация CRUD-интерфейса для управления столиками и бронированиями, включая загрузку изображений столиков и меню	7	29.05.2026	
Задача 3.4. Реализация системы проверки доступности столиков и управления временем бронирования с поддержкой экспорта отчётов о занятости столиков (CSV, PDF)	12	02.06.2026	
Задача 3.5. Разработка системы уведомлений и генерации подтверждений бронирования с вложением файлов (PDF-чеки)	6	06.06.2026	
Раздел 4. Оформление итогов работы			
Задача 4.1. Создание Git-репозитория с кодом проекта	3	10.04.2026	
Задача 4.2. Деплой приложения на хостинг	4	10.06.2026	
Задача 4.3. Оформление отчёта о проделанной работе	3	13.06.2026	

Руководитель курсовой работы (проекта):
старший преподаватель кафедры «Инфокогнитивные технологии»

«____» _____ 2026 г.

(подпись)

А. С. Кружалов

Дата выдачи задания «2» апреля 2026 г.

Дата сдачи выполненной работы «____» _____ 2026 г.

Задание принял к исполнению

«2» апреля 2026 г.



(подпись)

А. М. Антипова

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
Глава 1. Анализ предметной области	7
1.1 Обзор существующих программных продуктов по теме работы .	7
1.2 Анализ программных инструментов разработки веб-приложений	8
Глава 2. Проектирование веб-приложения	10
2.1 Анализ целевой аудитории	10
2.2 Описание функциональности приложения	10
2.3 Проектирование модели данных	12
2.4 Разработка макетов страниц	14
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	16

ВВЕДЕНИЕ

Актуальность темы. В условиях активной цифровизации сферы услуг автоматизация ключевых бизнес-процессов ресторанного бизнеса становится важным условием повышения качества обслуживания. Процесс бронирования столиков остаётся одним из наиболее трудоёмких в части ручного учёта: администраторы вынуждены вести резервации в журналах или электронных таблицах, что может приводить к ошибкам, двойным бронированиям и потере клиентов. Разработка специализированного веб-приложения, автоматизирующего цикл резервации столиков, является актуальной практической задачей.

Степень разработанности проблемы. На рынке существуют платформы бронирования и автоматизации ресторанного бизнеса, такие как Restoplace, Yclients и iiko [1—3]. Однако подобные решения могут быть избыточными для небольшого заведения, которому требуется простая и адаптируемая система управления бронированием. Поэтому разработка гибкого веб-приложения, легко адаптируемого под нужды конкретного ресторана, остаётся актуальной задачей.

Объект исследования — процесс управления бронированием столиков в ресторане.

Предмет исследования — программные средства и технологии разработки веб-приложений для автоматизации процесса резервации столиков, уведомления гостей и формирования отчётности.

Цель работы — разработка веб-приложения для автоматизации процесса бронирования столиков в ресторане, обеспечивающего управление резервациями, уведомление гостей и формирование отчётности.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести обзор существующих систем бронирования и анализ инструментов разработки с целью обоснования выбора технологического стека;
2. Проанализировать целевую аудиторию и описать функциональные требования к приложению посредством диаграммы вариантов использования и пользовательских историй (User Story);
3. Спроектировать модель данных: разработать диаграмму сущность-связь (ER-диаграмму), логическую и физическую схемы базы данных;

4. Разработать макеты страниц (Wireframe) и реализовать базовую структуру приложения с вёрсткой шаблонов страниц;
5. Реализовать систему аутентификации пользователей и интерфейс управления данными (CRUD) для столиков и бронирований, включая загрузку изображений;
6. Реализовать систему проверки доступности столиков и управления временем бронирования с поддержкой экспорта отчётов о занятости (CSV, PDF);
7. Разработать систему уведомлений и генерации PDF-подтверждений бронирования с отправкой гостям по электронной почте;
8. Создать репозиторий с исходным кодом проекта и выполнить развёртывание приложения на хостинге.

Методы исследования: анализ и сравнение программных продуктов, проектирование архитектуры веб-приложений, объектно-ориентированное программирование, прототипирование интерфейсов.

Практическая значимость работы состоит в том, что разработанное приложение может быть непосредственно внедрено в работу ресторана и заменить ручной учёт резервации столиков, сократив количество ошибок и повысив качество обслуживания гостей.

Глава 1. Анализ предметной области

1.1 Обзор существующих программных продуктов по теме работы

На современном рынке представлен ряд программных решений для автоматизации бронирования столиков в ресторанах. Для объективной оценки существующих систем были определены критерии, которым должно удовлетворять разрабатываемое приложение: онлайн-бронирование гостем без звонка в заведение; интерактивная схема зала с отображением статуса столиков; автоматическая проверка пересечений бронирований; отправка email-уведомлений с PDF-подтверждением; экспорт отчётов о занятости в форматах CSV и PDF; отсутствие значительных постоянных расходов; возможность адаптации логики под конкретное заведение.

В соответствии с перечисленными критериями рассмотрены три системы: **iiko**, **Restoplace** и **Yclients** [1—3].

Система **iiko** — российская платформа комплексной автоматизации ресторана (касса, кухня, склад, персонал). Управление схемой зала доступно администратору, однако онлайн-бронирование гостем не предусмотрено: резервации вносятся вручную сотрудником. Стоимость внедрения составляет от 50 000 рублей, что нецелесообразно для заведений, которым требуется только онлайн-бронирование.

Сервис **Restoplace** — отечественная облачная платформа, специализирующаяся на бронировании столиков. Гость самостоятельно выбирает столик на интерактивной схеме зала через виджет на сайте или во ВКонтакте. Бесплатный тариф ограничен 300 заявками в месяц. Принципиальным ограничением является отсутствие генерации PDF-подтверждений и экспорта отчётов — функций, требуемых в данном проекте.

Сервис **Yclients** — наиболее распространённая в России CRM-система для сферы услуг (более 26 000 подключённых заведений). Предоставляет виджет онлайн-записи, SMS/email-уведомления, интеграцию с кассами (АТОЛ, Эвотор). Однако система изначально ориентирована на запись к мастеру, а не на ресторанное бронирование: нативная схема зала отсутствует, подписка стоит от 3 000 рублей в месяц.

Результаты сравнения систем по выделенным критериям приведены в таблице 1.

Анализ показал, что ни одна из рассмотренных систем не удовлетворя-

Таблица 1 – Сравнительный анализ существующих систем по критериям

Критерий	iiko	Restoplace	Yclients	Собственная разработка
Онлайн-бронирование гостем	—*	+	+	+
Интерактивная схема зала	+	+	—	+
Проверка доступности столиков	+	+	+	+
Email-уведомления и PDF-подтверждения	—	—	—	+
Экспорт отчётов (CSV, PDF)	—	—	—	+
Доступность для малого бизнеса	—	+/—	+/—	+
Возможность кастомизации	—	—	—	+

* Бронирование вносится вручную сотрудником; самостоятельная онлайн-резервация гостем не предусмотрена.

ет всем критериям одновременно. Ключевыми незакрытыми потребностями являются генерация PDF-подтверждений и экспорт отчётов о занятости. Разработка собственного веб-приложения, полностью адаптированного к требованиям заведения, является обоснованной и целесообразной.

1.2 Анализ программных инструментов разработки веб-приложений

Перед выбором технологического стека проведён обзор инструментов, актуальных для современной fullstack-разработки, по четырём уровням.

Клиентская часть. Для построения интерфейсов веб-приложений широко применяются React, Vue.js и Angular [4]. React — библиотека, основанная на компонентном подходе, с наибольшей экосистемой. Vue.js прост в освоении, но уступает по объёму готовых библиотек. Angular ориентирован на крупные корпоративные проекты и требует значительного времени на конфигурацию. Для проекта выбран **React**: предоставляет готовые компоненты для интерактивных форм бронирования (react-calendar, react-toastify).

Серверная часть. Среди актуальных решений выделяются Node.js, Django, FastAPI и Spring Boot [5]. Node.js эффективен при большом числе соединений, но не имеет встроенных аутентификации и ORM. FastAPI обес-

печивает быструю разработку API, однако лишён административного интерфейса. Spring Boot масштабируем, но требует длительной конфигурации. Для проекта выбран **Django**: предоставляет встроенную аутентификацию, ORM и административную панель, что напрямую закрывает требования задания.

База данных. Наиболее распространены PostgreSQL, MySQL и SQLite [6]. SQLite не поддерживает параллельную запись и непригодна для production. MySQL уступает PostgreSQL по строгости транзакций. Для проекта выбрана **PostgreSQL**: обеспечивает надёжную обработку конкурентных запросов при одновременном бронировании одного столика, полностью совместима с Django ORM.

Среда развёртывания. Традиционная ручная настройка сервера создаёт риск расхождения окружений. Контейнеризация с помощью **Docker** и docker-compose устраняет эту проблему: все сервисы описываются в едином файле и воспроизводятся на любом хосте [7].

Глава 2. Проектирование веб-приложения

2.1 Анализ целевой аудитории

Разрабатываемое приложение ориентировано на два типа пользователей, взаимодействующих с системой с различными целями и уровнем доступа.

Гость — посетитель ресторана, желающий забронировать столик онлайн. Основная потребность: быстро выбрать удобное время и столик, получить подтверждение без необходимости звонить в заведение. Как правило, взаимодействует с приложением с мобильного устройства, ожидает простой и интуитивный интерфейс.

Администратор — сотрудник ресторана, управляющий бронированиями и информацией о столиках. Основные потребности: просматривать актуальный список резерваций, добавлять и редактировать столики, отмечать статус бронирований, формировать отчёты о занятости. Работает преимущественно с десктопного браузера в течение рабочей смены.

Для описания целевой аудитории применён метод персонажей (Personas), позволяющий формализовать потребности и сценарии поведения каждой роли [8]. Разграничение ролей реализуется на уровне системы аутентификации: гость имеет доступ только к форме бронирования и личному кабинету, администратор — к полной панели управления.

2.2 Описание функциональности приложения

Функциональные возможности системы определяются через взаимодействие ролей с её компонентами. Пользовательские истории (User Story) — короткие описания функциональности с точки зрения конкретного пользователя — широко применяются в гибких методологиях разработки для фиксации требований [9]. Основные пользовательские истории системы представлены ниже.

Для гостя:

- «Я, как гость, хочу зарегистрироваться и войти в систему, чтобы управлять своими бронированиями»;
- «Я, как гость, хочу выбрать свободный столик на нужную дату и время, чтобы гарантированно занять место в ресторане»;
- «Я, как гость, хочу получить письмо с PDF-подтверждением брони»;

вания, чтобы иметь доказательство резервации»;

- «Я, как гость, хочу отменить бронирование через личный кабинет, если мои планы изменились».

Для администратора:

- «Я, как администратор, хочу просматривать все бронирования на выбранную дату, чтобы планировать загрузку зала»;
- «Я, как администратор, хочу добавлять и редактировать столики (название, вместимость, фото), чтобы поддерживать актуальную информацию»;
- «Я, как администратор, хочу изменять статус бронирования (подтверждено, отменено, завершено), чтобы отражать реальное состояние зала»;
- «Я, как администратор, хочу экспортировать отчёт о занятости столиков в форматах CSV и PDF, чтобы анализировать загруженность заведения».

На рисунке 1 представлена диаграмма вариантов использования системы, построенная в нотации UML [10].

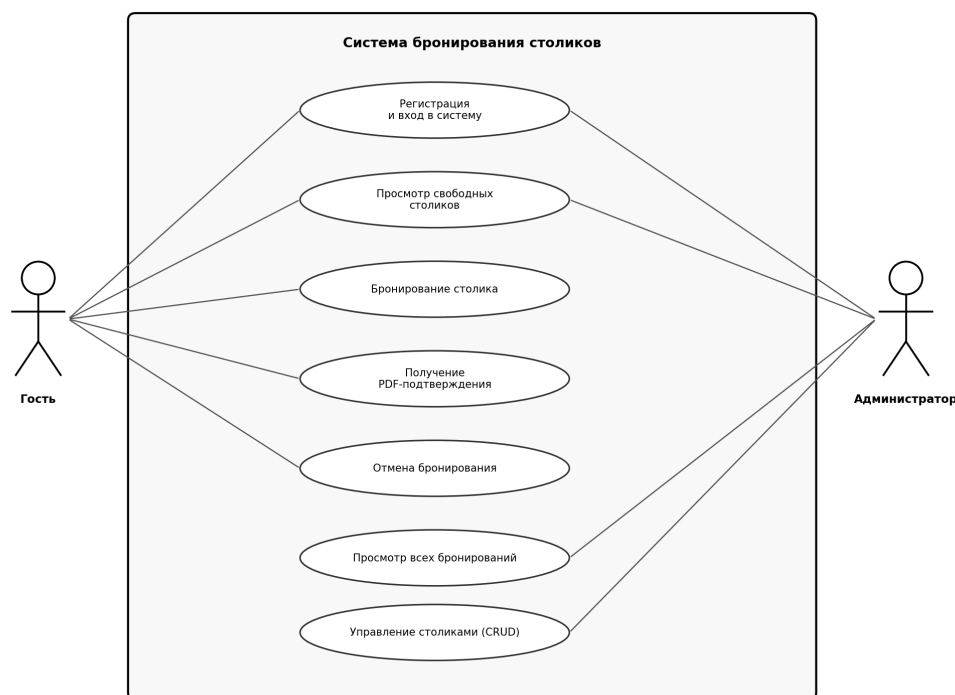


Рис. 1 – Диаграмма вариантов использования системы

Диаграмма отражает два актора — Гость и Администратор — и со-

вокупность вариантов использования, сгруппированных по функциональным блокам: управление учётной записью, бронирование столиков и административное управление.

2.3 Проектирование модели данных

Для хранения данных приложения спроектирована реляционная модель, включающая три основные сущности: пользователь (users), столик (tables) и бронирование (bookings). Метод диаграмм сущность-связь (Entity-Relationship model) позволяет наглядно описать структуру данных и отношения между сущностями [11]. Связи между сущностями отражены на ER-диаграмме (рисунок 2).

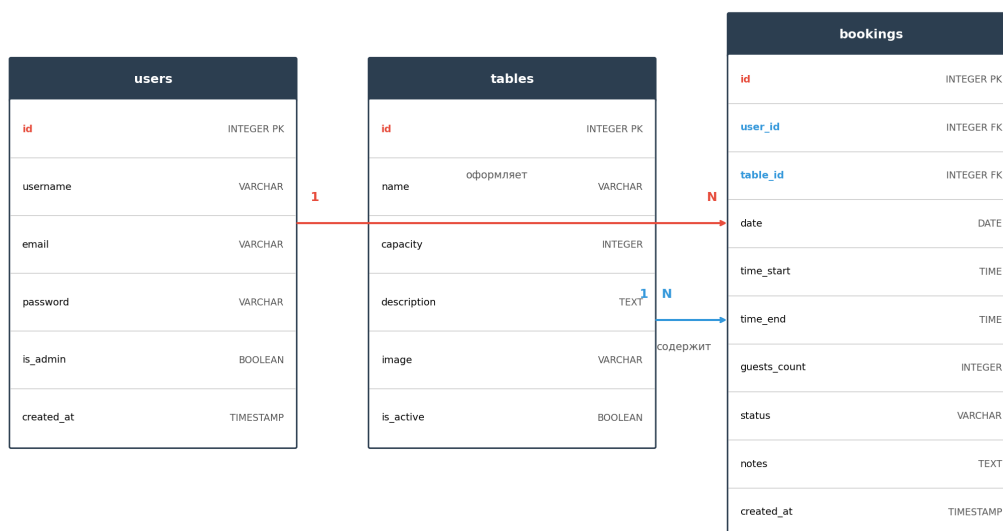


Рис. 2 – ER-диаграмма модели данных

Один пользователь может иметь несколько бронирований (связь «один ко многим»). Один столик также может фигурировать в нескольких бронированиях в разные временные интервалы (связь «один ко многим»). Пересечение бронирований по времени для одного столика исключается на уровне бизнес-логики сервера.

Физическая схема базы данных представлена тремя таблицами. Структура таблицы пользователей приведена в таблице 2.

Таблица 2 – Структура таблицы пользователей (users)

Поле	Тип	Ограничение	Описание
id	INTEGER	PRIMARY KEY, AUTO	Уникальный идентификатор пользователя
username	VARCHAR	NOT NULL, UNIQUE	Логин пользователя
email	VARCHAR	NOT NULL, UNIQUE	Адрес электронной почты
password	VARCHAR	NOT NULL	Хэш пароля (bcrypt)
is_admin	BOOLEAN	DEFAULT FALSE	Признак администратора
created_at	TIMESTAMP	DEFAULT NOW()	Дата и время регистрации

Структура таблицы столиков приведена в таблице 3.

Таблица 3 – Структура таблицы столиков (tables)

Поле	Тип	Ограничение	Описание
id	INTEGER	PRIMARY KEY, AUTO	Уникальный идентификатор столика
name	VARCHAR	NOT NULL	Название или номер столика
capacity	INTEGER	NOT NULL, CHECK>0	Максимальное количество гостей
description	TEXT	NULL	Описание (расположение, особенности)
image	VARCHAR	NULL	Путь к фотографии столика
is_active	BOOLEAN	DEFAULT TRUE	Доступность столика для бронирования

Структура таблицы бронирований приведена в таблице 4.

Таблица 4 – Структура таблицы бронирований (bookings)

Поле	Тип	Ограничение	Описание
id	INTEGER	PRIMARY KEY, AUTO	Уникальный идентификатор бронирования

Поле	Тип	Ограничение	Описание
user_id	INTEGER	FOREIGN KEY	Ссылка на таблицу пользователей
table_id	INTEGER	FOREIGN KEY	Ссылка на таблицу столиков
date	DATE	NOT NULL	Дата бронирования
time_start	TIME	NOT NULL	Время начала резервации
time_end	TIME	NOT NULL	Время окончания резервации
guests_count	INTEGER	NOT NULL, CHECK>0	Количество гостей
status	VARCHAR	DEFAULT 'pending'	Статус: pending, confirmed, cancelled, done
notes	TEXT	NULL	Комментарий гостя
created_at	TIMESTAMP	DEFAULT NOW()	Дата и время создания записи

2.4 Разработка макетов страниц

До начала реализации были разработаны низкодетализированные макеты (Wireframe) ключевых страниц приложения. Прототипирование интерфейса на этапе проектирования позволяет выявить и устранить структурные проблемы до написания кода, сокращая стоимость доработок [12]. Макеты позволяют определить структуру интерфейса и расположение элементов до написания кода.

Макет страницы бронирования представлен на рисунке 3. Страница содержит форму выбора даты (компонент react-calendar), времени, количества гостей и доступных столиков. После заполнения формы гость отправляет заявку и получает уведомление о результате.

Макет административной панели представлен на рисунке 4. Панель включает таблицу актуальных бронирований с возможностью фильтрации по дате и столику, кнопки изменения статуса и ссылки на экспорт отчётов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Restoplace: сервис бронирования столиков для ресторанов. — URL: <https://restoplace.cc/> (дата обр. 24.04.2026).
2. Yclients: облачная платформа автоматизации сервиса. — URL: <https://www.yclients.com/> (дата обр. 24.04.2026).
3. iiko: система автоматизации ресторанного бизнеса. — URL: <https://iiko.ru/> (дата обр. 24.04.2026).
4. React Documentation. — URL: <https://react.dev/> (дата обр. 24.04.2026).
5. Django Documentation. — URL: <https://docs.djangoproject.com/> (дата обр. 24.04.2026).
6. PostgreSQL Documentation. — URL: <https://www.postgresql.org/docs/> (дата обр. 24.04.2026).
7. Docker Documentation. — URL: <https://docs.docker.com/> (дата обр. 24.04.2026).
8. *Куликов С.* Тестирование программного обеспечения. Базовый курс. — Минск: Четыре четверти, 2017.
9. *Орлов С. А.* Технологии разработки программного обеспечения. — Санкт-Петербург: Питер, 2012.
10. *Леоненков А. В.* Самоучитель UML 2. — Санкт-Петербург: БХВ-Петербург, 2007.
11. *Кириллов В. В., Громов Г. Ю.* Введение в реляционные базы данных. — Санкт-Петербург: БХВ-Петербург, 2012.
12. *Раскин Д.* Интерфейс: новые направления в проектировании компьютерных систем. — Санкт-Петербург: Символ-Плюс, 2005.